

GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

Faculty of Computing, Department of Information Technology

ESSENTIALS OF ARTIFICIAL INTELLIGENCE IT3182

FINAL REPORT - GROUP 53

AI-Powered Emergency Blood Donor Recommendation and Prioritisation Decision Support System

Group Members

Student Number	**Student Name**
D/BSE/25/0026	HAMY Hettiarachchi
D/BCS/25/0026	RMSA Rathnayake
D/BIT/24/0079	WS Sanali Avihinsa
6935	AMSC Adhikari

Table of Contents

Abstract **3**
1 Introduction **4**
2 Problem Background **4**
3 Problem Statement **4**
4 Objectives **5**
5 Selected AI Techniques **5**
5.1 Pillar 1: Deductive AI (rule-based reasoning) 5
5.2 Pillar 2: Predictive AI (supervised classification) 6
5.3 Pillar 3: Decision AI (MCDA) and Explainable AI 6
6 Justification of the AI Techniques **6**
7 System Design **9**
7.1 Architecture 9
7.2 End-to-end workflow 9
7.3 Inputs and outputs 9
7.4 Datasets 11
7.5 Database schema 12
8 Implementation Details **13**
8.1 Technology stack and modules 13
8.2 Machine-learning implementation 13
8.3 Other implemented components 14
8.4 Dashboard modules 14
9 Testing and Results **15**
9.1 Test strategy 15
9.2 End-to-end test: O+ request at the National Hospital, Colombo 15
9.3 Eight emergency scenarios 16
9.4 Machine-learning evaluation 17
9.5 Feature importance and the deployment-population check 19
9.6 Explanation check 19
9.7 Interface testing and screenshots 20
10 Limitations and Discussion **22**
11 Ethical and Social Considerations **23**
12 Conclusion **23**
References **25**
Individual Contribution **26**

Abstract

The rapid identification of a compatible and willing donor is a consistent challenge during emergencies. Traditional hospital blood bank systems and voluntary registries in Sri Lanka normally operate via an unsorted list through the phone system, with no distance calculations, no automatic validation of the mandatory recovery period, and without any responsiveness. This research creates a decision support prototype using a combination of three artificial intelligence (AI) components; deductive reasoning for compatibility (ABO/Rh), availability and 90-day recovery period a random forest classifier to predict the likelihood of acceptance by eligible donors and multi-criteria decision analysis (MCDA) ranking with an XAI model. The system is created as a six-module Streamlit dashboard on a virtual registry with 1,400 donors in ten Sri Lankan cities.

The rule engine cuts down the registry from 1,400 donors to 394 eligible candidates for a sample O+ request in Colombo without using any model prediction and eight emergency scenarios involving all eight blood types of recipients were tested end-to-end in approximately 30 milliseconds each. In a held-out 20% of the registry the chosen Random Forest algorithm achieves an accuracy of 91.07%, recall of 96.18% and a ROC-AUC score of 0.9486 better than both Logistic Regression and a Decision Tree. A further check proves that these results are overly optimistic for the population scored by the model because availability and recency features form the key basis of the scoring while these have already been taken care of by the rule engine on eligible rule donors the ROC-AUC drops to 0.6925. The ranking has been validated for sample requests but has not yet been compared to other possible baselines and all data is synthetic.

Keywords: blood donor recommendation, rule-based reasoning, Random Forest, multi-criteria decision analysis, explainable AI, decision support.

1 Introduction

Blood transfusions represent an important part of the health care services especially in case of such emergencies as heavy traumas, obstetric problems, surgeries or other situations when there is a significant blood loss. According to the WHO patients who need transfusions do not always have access to safe blood and the well-developed blood system should be created.
In an emergency situation, the problem is not only in the existence of a certain blood group in the database. There are additional factors that the coordinator has to consider while choosing donors. It includes whether donors are compatible, available, sufficiently recovered from the previous donation, geographically accessible and willing to give blood. This task involves rule-making, forecasting, ranking and personal judgement.
Thus, the proposed system solves the problem of operational decision support not the clinical transfusion practices. Its goal is to save the time of a coordinator in choosing the most appropriate donor from the database. Therefore, the project combines AI technologies with clear procedures so that recommendations can be examined by an authorized coordinator.

2 Problem Background

In Sri Lanka, the identification of blood donors is done through donor lists of hospitals, telephoning, personal contact and posting online on social media. Such techniques can locate donors but take much time and effort and the donor contacted might not be available, far from the hospital or within the recovery period after his last donation.

The people concerned are patients awaiting blood transfusion, hospital blood bank personnel and coordinators, voluntary blood donor agencies and the donors themselves, who keep being contacted inefficaciously. Conventional registries can provide storage and retrieval facilities without any prioritization facility. With the help of AI, the donor search process can be made more efficient by considering multiple aspects of the donors together, such as blood type, availability, distance, past donation history and past response history.

3 Problem Statement

In an emergency, finding a qualified donor by manual search is slow and inefficient. Five concrete deficiencies were identified:

Table 1. Deficiencies in current practice.

Deficiency	**Operational consequence**
Sequential Latency: Unranked lists sorted alphabetically or chronologically based on registration	Contact cycles take place beyond the timeframe needed to make a transfusion decision
Geographic Invisibility: Address data available but no calculation of distances	Members living 40 km away are contacted before those living 2 km away but are equally well-matched
Ineligibility of Clinical Eligible Waste: No automatic check for recovery period	The calls go out to donors who would be ineligible for donation

Response-rate uncertainty: engagement history recorded but unused	Reliable donors are not preferentially reached
Opacity: ranking, where it exists, gives no justification	Coordinators cannot validate it under time pressure and revert to manual methods

There is need for an auxiliary decision support system, which will screen out ineligible donors compute their probability of responding and rank those remaining.

4 Objectives

1. **Deductive filtering:** use ABO/Rh blood type compatibility, eligibility and at least a 90 day time interval between donations.
2. **Predictive analytics:** predict the probability of a positive response from an eligible donor to an emergency appeal.
3. **Multi-criteria sorting:** sort eligible individuals by means of a transparent multi-criteria scoring mechanism that takes into account the response probability, availability, proximity and previous donation activity.
4. **Explainability:** generate intelligible explanations for the recommendations made based on evidence.
5. **Interactive delivery:** provide the pipeline through an interactive dashboard interface without need for programming knowledge.
6. **Evaluation:** test the pipeline on different emergency cases using simulated donor data.

Target users: authorized healthcare professionals and hospital staff (primary), blood donation organizations, system administrators and registered donors (availability updates). The tool is for medical staff use the patients do not choose the donors.

5 Selected AI Techniques

5.1 Pillar 1: Deductive AI (rule-based reasoning)

A deterministic forward-chaining production-rule engine (src/rule_engine.py, src/blood_compatibility.py) removes invalid candidates before any probabilistic reasoning.

Table 2. Rule base of the deductive engine.

Rule	Predicate	Basis
R1 Serology	blood_group is in Compatible(recipient), from an 8x8 ABO/Rh matrix	Incompatible red cells risk an acute haemolytic reaction
R2 Availability	available == 1	Opted-out donors waste triage time
R3 Safe interval	days_since_last_donation >= 90 (donors with no recorded donation pass)	Recovery period after whole-blood donation
R4 Fallback	If R3 empties a set that R2 did not, return the availability-filtered set instead of nothing	An empty list in an emergency is itself a failure (the relaxation is

		not flagged in the interface; see Section 10)

Examples: O- recipients accept only O- donors; A+ recipients accept O-, O+, A- and A+; AB+ recipients accept all eight groups. The engine returns the filtered candidates and an audit summary recording how many donors each rule removed.

5.2 Pillar 2: Predictive AI (supervised classification)

The binary classifier uses target_response (target_response = 1 if the donor replied) and gives output of probability using predict_proba. Three classifiers are tested on the same stratified dataset: Random Forest (with 120 trees, max_depth=8 and min_samples_split=4), L2 regularized Logistic Regression with feature standard scaling, and Decision Tree (max_depth=6). The features used include nine features: age, availability, number of donations, time since last donation, number of requests, number of responses, response rate, Haversine distance from donor to hospital, and contact_before. During inference, the distance feature gets recalculated based on the current request hospital location.

5.3 Pillar 3: Decision AI (MCDA) and Explainable AI

Each eligible donor is scored on four normalised sub-scores (0 to 100) combined by a weighted linear sum:

Score = 0.40 x S_resp + 0.30 x S_avail + 0.20 x S_prox + 0.10 x S_hist

- S_resp = predicted response probability x 100
- S_avail = 100 if the donor is available, otherwise 0
- S_prox = clip(100 - 10 x distance_km, 0, 100), so proximity falls to zero at 10 km
- S_hist = clip(10 x donation_count, 0, 100), saturating at 10 lifetime donations

Scores map to four tiers: Highly Recommended (90 or more), Recommended (75 to 89.9), Moderately Recommended (60 to 74.9) and Low Priority (below 60). Ties are broken by score, then response probability, then shorter distance. The XAI module produces six evidence bullets per donor (compatibility, availability, proximity, response history, model probability, donation history and recovery interval). Each sentence is generated from an observed value with thresholded wording (for example 3 km or less is "exceptional proximity"), and Section 9.6 checks the wording against observed donor values.

6 Justification of the AI Techniques

Rule-based reasoning is appropriate where the needs are definite and fixed. The blood compatibility rules and recovery intervals are established by medical knowledge, not by learning from examples; hence, using a probabilistic model would be inappropriate and unreliable. Rules are completely interpretable; users know what rule dropped out each donor.

Machine-learning classification is appropriate for the only need that cannot be stated in a rule – whether a certain donor will respond to a certain request. Patterns in historical behaviour (response rate, number of donations, recency) and proximity to the hospital can be discovered in structured data by a classifier. Random Forest, Decision Tree and Logistic Regression are standard algorithms for tabular data, are measured using standard metrics, and tree-based models have the added advantage of showing feature importances. The probability rather than class itself is needed for ranking downstream, hence ROC-AUC (Section 9.4) is a better metric than accuracy.

A need for MCDA with XAI arises from the fact that there are multiple viable donors at hand, and the coordinator has to select one. A weighted linear score enables the balancing of conflicting criteria (e.g., the somewhat unresponsive but much more proximate donor), can be precisely decomposed by criteria, and hence admits explanations. The reason why explainability is crucial is the fact that clinicians will never accept an opaque score under time constraints. The three methods together build up a reasoning-prediction-recommendation pipeline.

The following table lists the alternatives that were considered in this analysis for each of the three components and the reasons why the particular method was selected. The references are made to the empirical studies reported in this paper; the alternatives that were not implemented (neural network, Naive Bayes, SHAP, LIME) were evaluated solely on design criteria.

Table 3. Technique choices and alternatives considered.

Component	**Chosen technique**	**Alternative considered**	**Justification and evidence**
Safety filtering	Deductive rule engine	Distance-based k-nearest neighbours or unfiltered machine learning	Deterministic rules for red-cell compatibility and recovery period are applied to reduce the risk of a ranking error using the statistical model (Section 9.3).
Response likelihood	Random Forest, tuned by GridSearchCV	Neural network (MLP), Naive Bayes; Logistic Regression and Decision Tree were also benchmarked	Data with non-linear relationships and requirement for probability values. Random Forest gave the highest ROC-AUC and most consistent cross-validation among all three benchmarked models.

Prioritisation	MCDA weighted score	Single-criterion sorting (distance only or response history only)	Distance alone, and response history alone, each neglect the other variable. The balance between the two variables is evident in the scoring analysis (Section 9.2); no quantitative comparison using a single criterion was made (Section 10).
Interpretability	Rule-based natural-language explanations	SHAP (Lundberg & Lee, 2017) or LIME surrogate plots	Coordinators in a hurry require concise operational sentences. The SHAP values represent feature importance with respect to the model but not with respect to the donor. The sentences are derived using donor values observed (Section 9.6). SHAP was not implemented.

7 System Design

7.1 Architecture

The system has four layers. The presentation layer is a Streamlit dashboard (app.py) with six modules: Dashboard, Emergency Request, Recommended Donors, Donor Database, Model Performance and About System. The orchestration layer controls the pipeline, session state and caching of the model and database handles. The intelligence layer contains the rule engine, the preprocessing and prediction modules, and the ranking and XAI module. The persistence layer is a SQLite database with four tables (donors, emergency_requests, predictions, donation_history) plus serialised model artefacts.

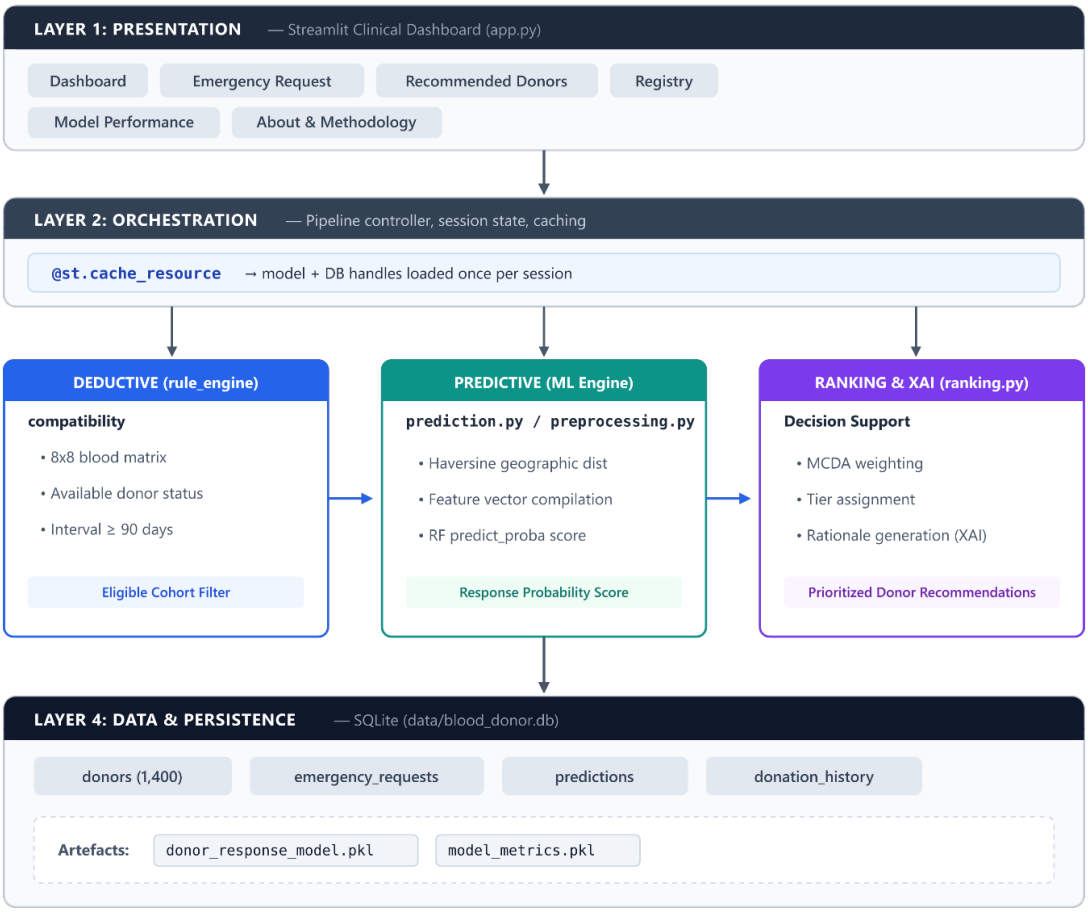


Figure 1. Layered architecture of the system.

7.2 End-to-end workflow

1. An authorised user submits a requisition: blood group, hospital coordinates, units, urgency, date and time.
2. The donor registry is retrieved from the database.

3. **Stage 1, deductive filtering:** rules R1 to R4 are applied and an attrition audit is recorded.
4. **Stage 2, feature engineering and inference:** the Haversine distance to the hospital is computed, a feature vector is assembled for each candidate, and the Random Forest outputs P(response).
5. **Stage 3, MCDA ranking and explanation:** scores, ranks, tiers and rationale bullets are generated.
6. The top-N donors are displayed with explanation cards and logged to the predictions table for audit.
7. The healthcare coordinator contacts donors and confirms medical suitability. The system never makes the final decision.

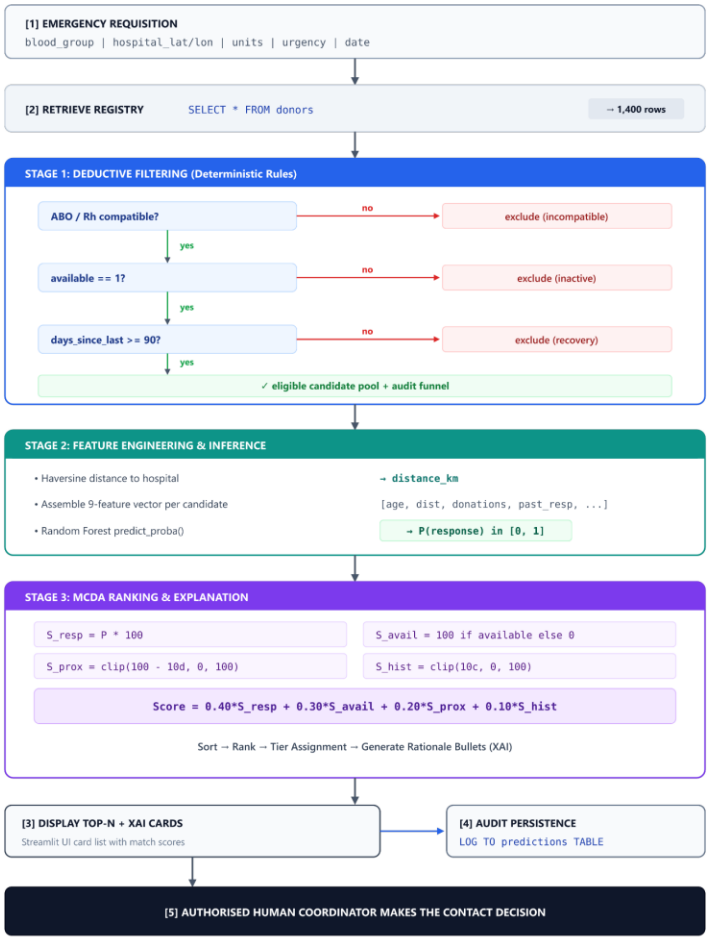


Figure 2. End-to-end request workflow.

7.3 Inputs and outputs

Table 4. System inputs and outputs.

Inputs	Outputs
Requisition: blood group, hospital location, units, urgency, date and time	Filtered eligible donor list with a rule-attrition audit
Donor record: ID, age, blood group, coordinates, availability, donation count, days since last donation, request and response history	Distance from each candidate to the hospital
Synthetic historical outcomes for model training	Response probability, recommendation score and tier per donor
Optional override of the safe interval (30 to 180 days)	Top 3, 5 or 10 donors with an evidence-based explanation

7.4 Datasets

Donor data are purposely not employed: this sort of health information cannot possibly be sourced legally or ethically for an undergraduate project. The data in use is fabricated, and all outcomes presented here in the report pertain to the prototype using simulated data.

Donor registry (data/donors.csv, 1,400 donors, 15 columns, random_state = 42). Blood groups follow an approximate South Asian profile (Figure 3), with AB- genuinely scarce (11 donors). Donors are located in ten Sri Lankan centres: Colombo 420, Kandy 231, Gampaha 190, Negombo 112, Kalutara 110, Galle 105, Kurunegala 83, Matara 54, Ratnapura 49 and Anuradhapura 46. Responders are 786 (56.1%) and non-responders 614 (43.9%); 1,013 donors (72.4%) are available; 740 are male and 660 female (gender is held for reporting and excluded from the model). Ages are 18 to 64 (mean 32.7), donation counts 1 to 25 (mean 4.10) and days since last donation 30 to 449.

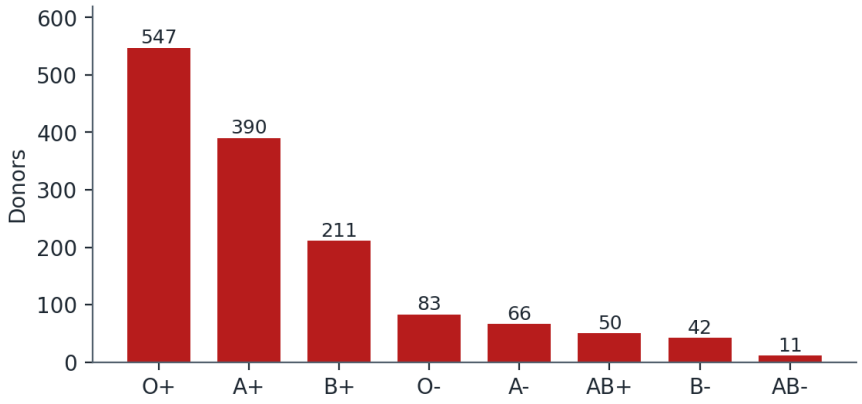


Figure 3. Blood group composition of the donor registry (n = 1,400).

How the labels were generated. Reading the generator code shows that target_response is drawn from a logistic function of availability, response rate, donation count, a recency term, age and Gaussian noise, with 96% of unavailable donors forced to non-response. Consequently the

response rate is 0.3% for unavailable donors against 77.5% for available ones, and 8.2% below 90 days since the last donation against 64.5% above. Distance does not enter the label. This explains the feature-dominance finding in Section 9.5.

> **Synthetic data.** The model learns patterns that the generator defines, so the reported metrics show that the pipeline works, not how real donors behave.

7.5 Database schema

Persistence uses SQLite (data/blood_donor.db) with four tables, created in src/database.py.

Table 5. SQLite tables.

Table	Columns	Purpose
donors	donor_id (primary key), blood_group, age, gender, city, latitude, longitude, available, donation_count, days_since_last_donation, previous_requests, previous_responses, response_rate, contacted_before, target_response	Registry of 1,400 synthetic donor profiles
emergency_requests	request_id (primary key), required_blood_group, hospital, latitude, longitude, units_required, priority, request_date, notes	Log of every requisition submitted through the dashboard
donation_history	donation_id (auto-increment key), donor_id (foreign key to donors), donation_date, response, request_id	Longitudinal donation and response events
predictions	prediction_id (auto-increment key), request_id, donor_id (foreign key to donors), response_probability, recommendation_score, recommendation_label, created_at	Audit trail of every recommendation produced

8 Implementation Details

8.1 Technology stack and modules

The prototype is written in Python with Pandas and NumPy for data handling, Scikit-learn for modelling, Streamlit for the interface, SQLite for persistence and Matplotlib for charts. It was developed in Visual Studio Code and Jupyter with Git for collaboration (requirements.txt lists the dependencies).

Table 6. Module responsibility map.

Module	Responsibility
app.py	Six-page Streamlit interface (about 940 lines), routing, caching and theming
src/blood_compatibility.py	Bidirectional ABO/Rh matrices, compatibility predicate and text description
src/rule_engine.py	RuleEngine class and filter_donors: rules R1 to R4 and an attrition audit
src/preprocessing.py	Vectorised Haversine distance, feature assembly and imputation defaults
src/train_model.py	Three-model training, stratified split, cross-validation and model serialisation
src/prediction.py	Model loading and batch probability inference (predict_candidates)
src/ranking.py	DonorRanker, sub-scores, tiers and the XAI explanation generator
src/database.py	SQLite interface for the four tables, request and prediction logging
src/data_generator.py	Seeded synthetic registry generator
notebooks/model_training.ipynb	Exploration of the model training
verify_pipeline.py	Automated end-to-end integration test
scenario_tests.py, deployment_check.py, ui_smoke_test.py	Additional test scripts used for Section 9 (scenarios, deployment-population check, interface smoke test)

8.2 Machine-learning implementation

Features and split. The nine features are listed in Section 5.2. The registry is split 80/20 with stratification on the target (1,120 training and 280 test donors, random_state = 42), and each model is also scored with 5-fold stratified cross-validation on F1. At training time distance_km is the Haversine distance from each donor to a fixed reference point (National Hospital Colombo), while at inference it is recomputed for the hospital in the request.

Model selection. The train.py script always saves the model that has the best F1 score on the test set; the Random Forest model is evaluated first and replaces itself on a strictly better result only.

Random Forest and the Decision Tree model have the same F1 score (0.9235); hence, Random Forest is chosen. The evidence of the Random Forest superiority is given by the metrics that differentiate between the two models, which are ROC-AUC (0.9486 against 0.9377) and variance of the cross-validation score, explained in Section 6.

8.3 Other implemented components

- **Soft fallback.** When the 90-day rule does not yield any results but there are donors, the rule engine will give back the result from the availability filter rather than return an empty result set. This is important for rare blood types such as AB-.
- **Cold start.** The donor with no history at all (0 past requests) will have a response rate of 0/0. The preprocessing assigns an initial prior of 0.50 and uses default values for missing values. Since there is history for every record in the present registry, this code path has not yet been tested on any new donors (Chapter 10).
- **Explanation generator.** Six evidence bullets are produced per donor from observed values with thresholded wording: compatibility, availability, proximity (3 km or less is "exceptional", up to 10 km "rapid transit range"), response history, model probability, and donation history with days since the last donation.
- **Performance.** The model, database handle and metrics are cached with @st.cache_resource so that Streamlit does not reload them on every widget interaction.
- **Persistence.** Every requisition and every set of recommendations made are recorded into SQLite.

8.4 Dashboard modules

Table 7. Streamlit dashboard modules.

Module	Functionality
Dashboard	KPI cards (registry size, blood-group coverage, availability, average response rate); blood group, city and availability charts
Emergency Request	Requisition form: blood group, hospital coordinates, units, urgency, configurable minimum safe interval; writes to emergency_requests
Recommended Donors	Five-stage funnel, ranked candidate table, per-donor XAI evidence card with weighted score breakdown, mock SMS and call buttons
Donor Database	Searchable and filterable registry browser, add and edit donor records, CSV export
Model Performance	Comparative metric tables, confusion matrix and ROC-AUC summary from the saved metrics
About System	Methodology notes, MCDA equations, clinical disclaimer and limitations

9 Testing and Results

9.1 Test strategy

Table 8. Tests performed.

Test	What it checks	Section
End-to-end integration (verify_pipeline.py)	Database, rules, inference, scoring and explanation work together	9.2
Eight emergency scenarios (scenario_tests.py)	All eight recipient blood groups at different hospitals, including rare groups; runtime	9.3
Model evaluation	Three models on a stratified hold-out with cross-validation	9.4
Feature importance and deployment-population check (deployment_check.py)	Whether the metrics hold for the donors that actually reach the model	9.5
Explanation check	Whether the experience wording is supported by the observed donation count	9.6
Interface smoke test (ui_smoke_test.py)	All six pages load; the requisition form returns a ranked list	9.7

9.2 End-to-end test: O+ request at the National Hospital, Colombo

Sample input: blood group O+, National Hospital of Sri Lanka (6.9271 N, 79.8612 E), 2 units, critical urgency. The automated test verify_pipeline.py passed.

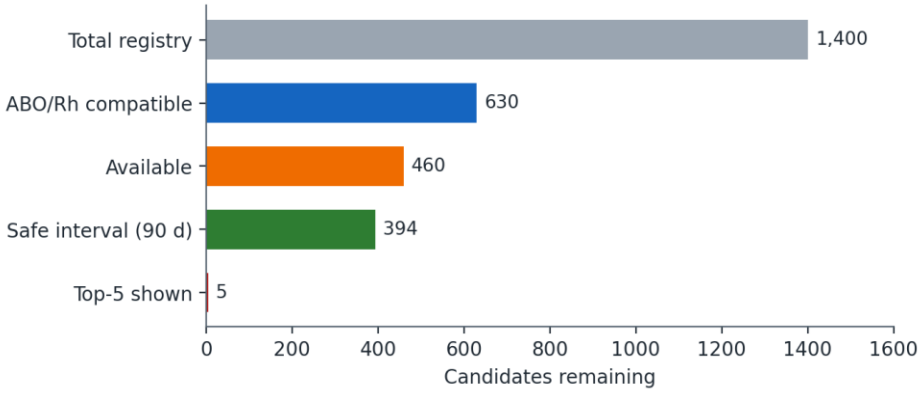


Figure 4. Rule-engine attrition for the O+ request at Colombo: 1,400 donors to 394 eligible candidates (71.9% removed before any inference).

Table 9. Top five donors returned for the O+ request at Colombo.

Rank	Donor	Distance	P(response)	Donations	Score	Tier
1	D0149	1.27 km	96.8%	9	95.2	Highly Recommended
2	D0083	2.11 km	98.2%	10	95.1	Highly Recommended
3	D0791	2.06 km	97.3%	11	94.8	Highly Recommended
4	D0608	1.14 km	97.0%	8	94.5	Highly Recommended

5	D0899	2.59 km	99.0%	17	94.4	Highly Recommended

Explanation generated for the top donor (D0149):

- Identical match (O+ to O+)
- Currently active and marked available for emergency dispatch
- Exceptional proximity: only 1.27 km from National Hospital Colombo
- Historical response record: 44% (8/18)
- 96.8% ML-predicted probability of affirmative emergency response
- Experienced donor (9 prior donations; 409 days since last donation)

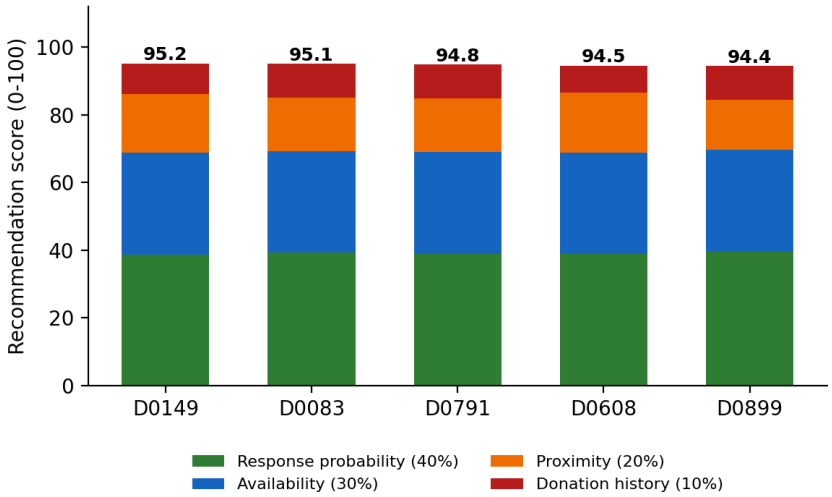


Figure 5. Contribution of each weighted criterion to the score of the top five donors.

The five scores lie within 0.8 points, which is where compensatory reasoning matters. D0899 has the highest response probability (99.0%) and the deepest history (17 donations) but ranks fifth: D0149 is closer, and its proximity advantage (17.46 against 14.82 weighted points, a gain of 2.64) outweighs the combined 1.88-point deficit on response probability and history. A sort on response probability alone would have reversed this order.

9.3 Eight emergency scenarios

To test the pipeline beyond a single request, one emergency was run for each recipient blood group at a different hospital, with the default 90-day interval. For every scenario the table gives the funnel, the best donor and the quality of the top five.

Table 10. Pipeline results for eight emergency scenarios.

Recipient	Hospital	Compat.	Avail.	Eligible	Top donor (dist., P, score)	Top-5 mean distance

O+	NH Colombo	630	460	394	D0149 (1.27 km, 96.8%, 95.2)	1.8 km
A+	TH Kandy	1086	789	668	D0452 (1.05 km, 94.9%, 95.9)	1.6 km
B+	TH Karapitiya (Galle)	883	640	552	D0263 (1.73 km, 95.0%, 94.5)	2.5 km
O-	DGH Negombo	83	67	55	D0567 (3.02 km, 93.6%, 82.4)	43.2 km
AB-	NH Colombo	202	150	126	D0720 (3.09 km, 97.8%, 92.9)	2.6 km
AB+	TH Anuradhapura	1400	1013	864	D0814 (3.00 km, 99.0%, 93.6)	1.7 km
B-	GH Matara	125	96	79	D0573 (2.23 km, 96.4%, 85.1)	37.9 km
A-	TH Kurunegala	149	113	94	D0453 (0.65 km, 88.9%, 94.3)	2.0 km

Observations. (1) The pipeline returned a ranked list for all eight groups and never an empty result; eligible pools range from 55 (O-) to 864 (AB+). (2) For common groups in dense areas (O+, A+, B+, AB-) the top five are within about 2 to 3 km of the hospital and all or nearly all "Highly Recommended". (3) For scarce groups the system degrades sensibly: for O- at Negombo and B- at Matara the best donor is only 3.0 km and 2.2 km away, but the top-five mean distance rises to 43.2 km and 37.9 km, no donor in the top five reaches the "Highly Recommended" tier, and the lowest top-five scores are 78.5 and 78.0. This is the information a coordinator needs: few close donors exist and a wider search is required. (4) The full pipeline (filtering, inference and ranking) takes about 30 ms per request on average over ten repeated O+ runs (maximum 34 ms), which suits emergency use.

9.4 Machine-learning evaluation

Table 11. Model benchmark on the held-out test set (n = 280, stratified 80/20 split).

Model	Accuracy	Precision	Recall	F1	ROC-AUC	5-fold CV F1
Random Forest (selected)	91.07%	88.82%	96.18%	0.9235	0.9486	0.9303 +/- 0.0042
Decision Tree	91.07%	88.82%	96.18%	0.9235	0.9377	0.9192 +/- 0.0124
Logistic Regression	89.64%	88.10%	94.27%	0.9108	0.9273	0.8957 +/- 0.0046

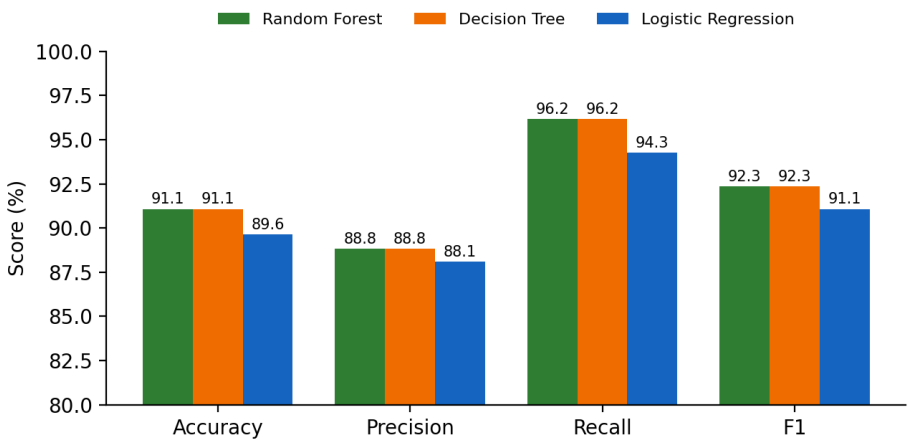


Figure 6. Threshold-based metrics for the three models on the held-out test set.

Random Forest and the Decision Tree give identical point metrics on this partition, so they are separated by ranking quality and stability: ROC-AUC 0.9486 against 0.9377, and a cross-validation standard deviation of 0.0042 against 0.0124 (about one third). Because the system uses the probability for ranking rather than a hard label, ROC-AUC is the more relevant measure and favours Random Forest. Logistic Regression is the weakest on every metric but still close, which suggests the signal in the data is largely simple.

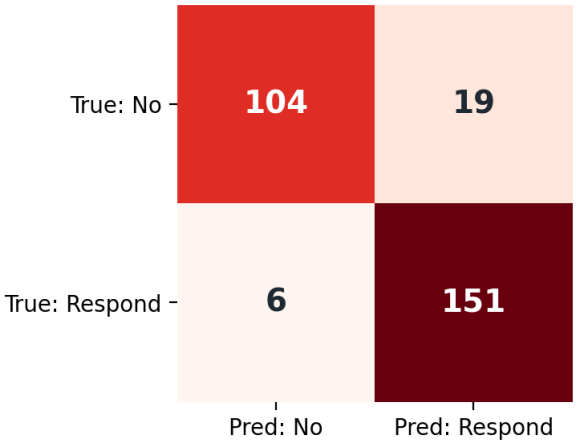


Figure 7. Random Forest confusion matrix on the 280-donor test set.

The confusion matrix shows 104 true negatives, 19 false positives, 6 false negatives and 151 true positives. A false positive costs one wasted call, while a false negative means a willing donor is under-prioritised, so the high recall (96.18%) suits an emergency setting.

9.5 Feature importance and the deployment-population check

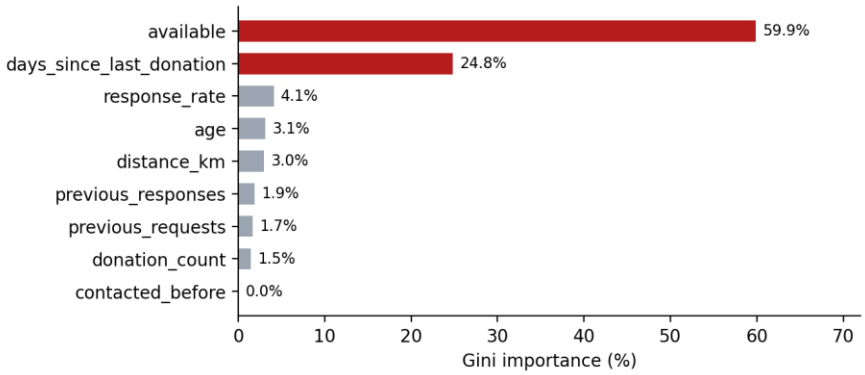


Figure 8. Gini feature importance of the selected Random Forest.

Two features account for 84.7% of the model: available (59.9%) and days_since_last_donation (24.8%). Both are hard rules that the rule engine applies before the model is used, so for the donors the model actually scores they are constant or close to it. contacted_before has zero importance because every record has the value 1, and distance_km has only 3.0% because the training distance is a static value. The headline metrics were therefore measured on the full registry, including donors who would never reach the model. To quantify the effect, the saved model was scored on the part of the same test split that passes the rule engine (available and at least 90 days since the last donation).

Table 12. Saved model scored on all test donors and on rule-eligible test donors only.

Test population	n	Positive rate	Accuracy	Precision	Recall	F1	ROC-AUC
All test donors (as reported)	280	56.1%	91.07%	88.82%	96.18%	0.9235	0.9486
Rule-eligible test donors only	172	88.9%	87.21%	88.76%	98.04%	0.9317	0.6925

On eligible donors the ROC-AUC falls from 0.9486 to 0.6925, and the accuracy (87.21%) is below the 88.95% obtained by always predicting "will respond", because 88.95% of eligible donors respond in this data. The model is still useful for ordering donors, but its headline accuracy overstates its discriminating power on the deployed population. The most direct remedy is to retrain on rule-eligible donors with request-specific distance, which is the main item of future work (Section 12).

9.6 Explanation check

The explanation generator was run for the top five donors of each of the eight scenarios (40 explanations). Numbers, distances, probabilities and availability statements are copied from observed values. However, the last bullet describes every donor as "Experienced donor (n prior donations ...)" whatever the count. In 9 of the 40 explanations the donor had fewer than five prior donations (O-: 2, AB+: 1, B-: 3, A-: 3 of 5), so the word "experienced" is not supported by the data. Five donations was taken here as a reasonable minimum for "experienced"; a banded wording

(for example "limited history" for 1 to 4 donations) would remove the problem and is listed as an improvement.

9.7 Interface testing and screenshots

ui_smoke_test.py uses Streamlit's AppTest to load each of the six modules and to submit the default requisition form. All six pages loaded with no exceptions. Submitting the form returned the message "Analysis complete! Found 394 eligible candidates. Displaying top 5 recommendations", one ranked table and five explanation cards, with no error messages. Figures 9 to 12 show the interface.

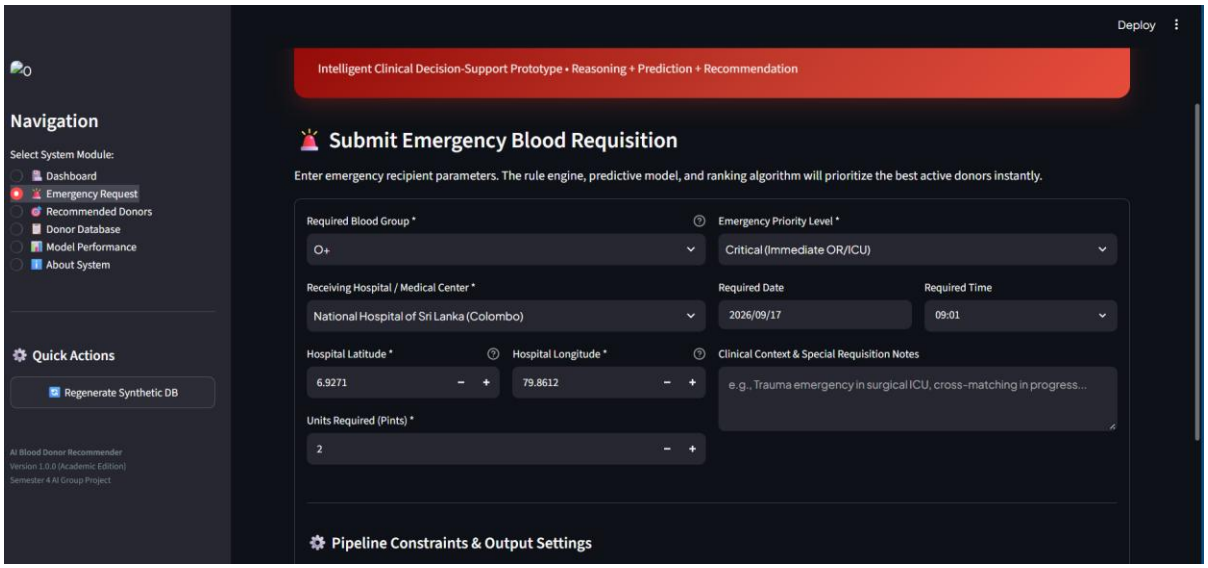


Figure 9. Emergency Request module: requisition form.



Figure 10. Dashboard module: registry KPIs and distributions.

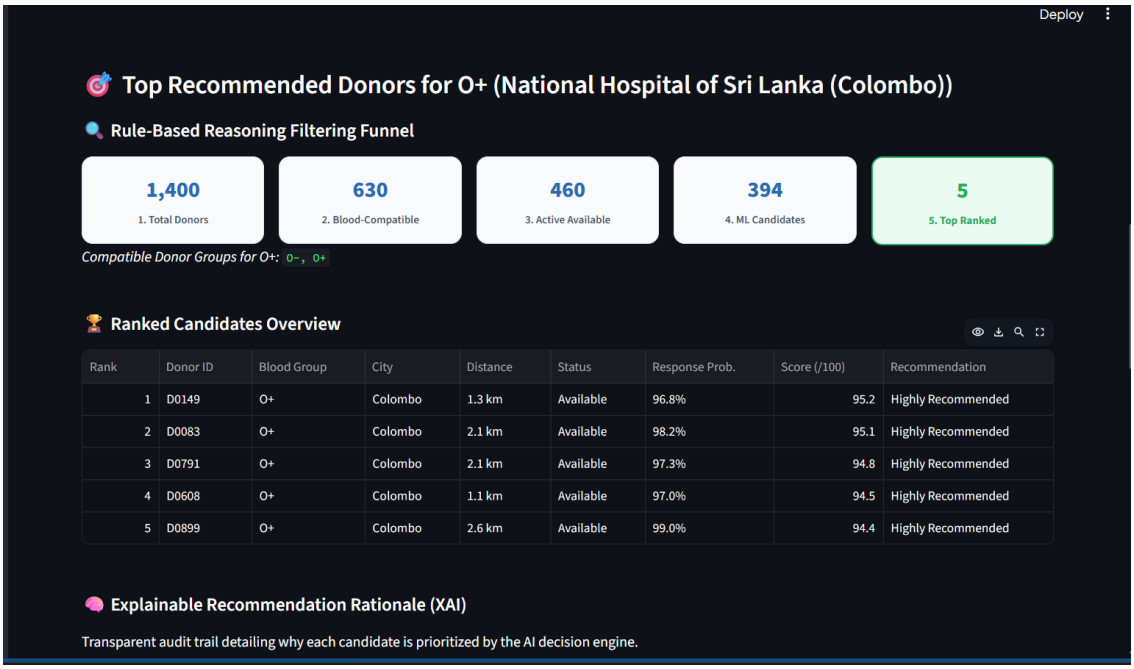


Figure 11. Recommended Donors module: rule-based filtering funnel and ranked candidates.

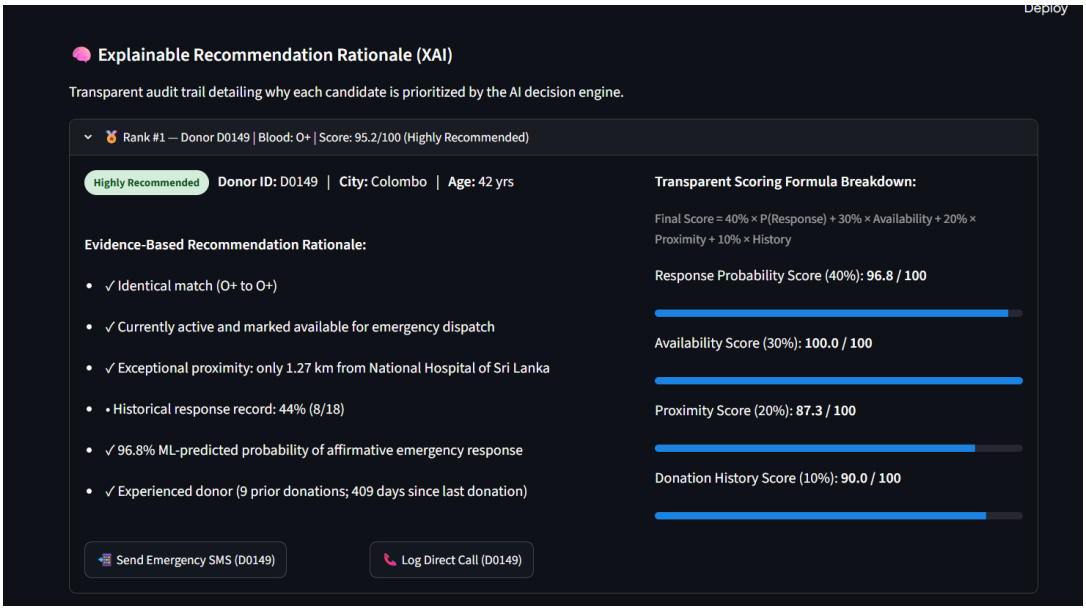


Figure 12. Explainable recommendation rationale with weighted score breakdown for the top donor.

Not tested. The ranking has not been benchmarked against baseline orderings (such as nearest-first or registry order), the weights have not been analysed for sensitivity, and no usability study with participants has been carried out. These gaps are discussed in Section 10.

10 Limitations and Discussion

While the prototype highlights how the three AI methods work together to deal with simulated emergency situations, the findings should be interpreted in light of certain limitations.

- **Synthetic data.** The results are based on all simulated donors where labels were created by the team. Metrics indicate the success of the pipeline, and do not mean that real donors act in the same way.
- **Optimistic headline metrics.** The 91.07% accuracy was measured on the whole registry. The model depends mainly on available and days_since_last_donation, which the rule engine fixes before prediction, and on rule-eligible donors the ROC-AUC is 0.6925 (Section 9.5).
- **Distance is barely learned.** The distance to the reference point is a static figure and does not affect the class label; hence, it is accorded minimal consideration by the model (3.0%). The proximity factor impacts the ranking through the manually set MCDA weight.
- **Ranking not benchmarked.** The project suggests that a ranked list is superior to an unranked list, yet no comparison with baselines has been carried out. The eight scenarios exhibit acceptable behavior, rather than superiority.
- **Hand-set MCDA weights.** Weights (0.40, 0.30, 0.20, 0.10) and saturation points (10 km, 10 donations) have been selected through judgement without carrying out any sensitivity analysis. After the rule engine, all candidates are eligible, hence the 30% availability criterion will give every individual an equal additional 30 points, increasing the score above the thresholds.
- **Explanation wording.** Every donor is described as "experienced" regardless of donation count (Section 9.6).
- **Silent fallback.** If the 90-day rule empties the candidate list, the engine relaxes it without flagging the affected donors in the interface. A visible supervisor-override flag is needed.
- **Cold start untested.** The 0.50 prior for donors without history is implemented but untested on genuinely new donors, and it is above the registry mean response rate of 0.378.
- **Distance approximation.** Haversine distance is shorter than road distance. The order of donors is largely preserved but travel times are not modelled.
- **Unvalidated medical rules.** The compatibility test and 90 day rule are in accordance with standard advice (such as the WHO donor selection criteria) and must be medically verified. Haemoglobin, infection screening, medication, travel history, or pregnancy status is not checked by the system.
- **Simulated dispatch.** The SMS and call buttons only display a confirmation message.
- **No usability evaluation.** The interface has been tested for correctness but not with users.

11 Ethical and Social Considerations

- **Data Privacy**. The donor's health and geographical data are sensitive. In the prototype, synthetic data with no personal information has been used. The live system requires explicit consent of the donor, data minimization, role-based access for authorized personnel only, encryption, logging (predictions table is one of them) and full compliance with Sri Lankan data protection laws.
- **Bias and Fairness.** Prioritization by response rate and geographical proximity can lead to favoring donors from well served urban regions while discriminating against new donors and those from rural regions. Gender is not considered in the algorithm, and a new donor is assigned a neutral value and not zero. Still, a routine audit of the recommended vs never contacted donors is necessary.
- **Accuracy and Reliability.** Uncertain predictions and the inflated claim of the headline accuracy (see Section 9.5) require that the interface present probabilities and tiers, not certainty, and even the highest ranked donor could choose not to donate.
- **Transparency.** Each recommendation comes with score details and six bullets of evidence to be challenged by the staff member. The wording should be accurate, and thus "experienced donor" term, which is way too optimistic, is considered an issue.
- **Human Responsibility and Automation Bias.** The system is a decision support system, hence the final medical eligibility determination and donor contact and consent must be made by qualified healthcare professionals. The list ranks and can foster over-reliance on the system, which is why the user interface includes the disclaimer statement.
- **Misuse.** Donor location and contact information can be used to harass donors or for commercial purposes, and the ranking itself could influence the donors. Hence, access control and logging decrease this threat.
- **Donor Autonomy and Fatigue.** Donors have an option to set their availability status. Contacting top-ranked donors repeatedly can cause donor fatigue, hence the need to monitor contact rate.

12 Conclusion

In this work, we have obtained a functional prototype which merges together deductive reasoning, supervised prediction and multi-criteria ranking with explanations for selecting emergency blood donors. The rule engine filters out the clinically incorrect donors without using any model (from 1,400 to 394 in the sample request), the Random Forest predicts the probability of the positive response with the ROC-AUC value of 0.9486 for the test sample, and the MCDA component outputs hierarchical and explained ranked lists via the dashboard which consists of six functional pages with no errors in execution. We have tested eight cases for all recipient groups and obtained the results in 30 ms each time without getting an empty list in the result even for rare groups.

The limitations revealed through the evaluation include those clearly expressed. The model relies on features which have been defined by the rule engine, and therefore the impressive ROC-AUC 0.9486 in its performance headline is exaggerated when evaluated among the donors it has scored (ROC-AUC 0.6925). Whether ranking is better than an unranked registry has not been evaluated, MCDA weights have not undergone sensitivity testing, relaxed interval fallback has not been marked in the interface, and the wording overstates experience. Data used is entirely synthetic, and usability testing has not been conducted.

Future work: Retrain model on eligible donors based on request-based distances; benchmark ranking against randomly ordered, distance-first ordered and response rate-first orders on simulated emergencies; conduct sensitivity tests of the weights; flag the relaxed-interval fallback clearly; modify experience wording according to donation frequency; test cold-start algorithm on new donors; incorporate road network routing and real dispatch; validate using real data and expert opinion; and evaluate usability using the System Usability Scale.

References

[1] “Blood donor selection: guidelines on assessing donor suitability for blood donation,” *www.who.int*. https://www.who.int/publications/i/item/9789241548519 (accessed Oct. 08, 2026).

[2] World Health Organization, “Blood safety and availability,” *Who.int*, May 30, 2025. https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability (accessed Oct. 08, 2026).

[3] “UCI Machine Learning Repository,” *archive.ics.uci.edu*. https://archive.ics.uci.edu/dataset/176/blood+transfusion+service+center (accessed Oct. 08, 2026).

[4] I.-C. Yeh, K.-J. Yang, and T.-M. Ting, “Knowledge Discovery on RFM Model using Bernoulli Sequence.” Elsevier BV, 2025. doi: 10.2139/ssrn.5398921.

[5] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[6] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python Gaël Varoquaux Bertrand Thirion Vincent Dubourg Alexandre Passos PEDREGOSA, VAROQUAUX, GRAMFORT ET AL. Matthieu Perrot Edouard Duchesnay,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011, Accessed: Oct. 08, 2026. [Online]. Available: https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf

[7] A. Géron, “Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition,” *O’Reilly | Safari*, 2019. https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/ (accessed Oct. 08, 2026).

[8] F. Doshi-Velez and B. Kim, “Towards A Rigorous Science of Interpretable Machine Learning,” *arXiv:1702.08608 [cs, stat]*, vol. 2, no. 2, Mar. 2017, Accessed: Oct. 08, 2026. [Online]. Available: https://arxiv.org/abs/1702.08608

[9] T. Lopes, “USE OF ARTIFICIAL INTELLIGENCE IN BLOOD DONATION: A LITERATURE REVIEW,” *Hematology, Transfusion and Cell Therapy*, vol. 43, p. S355, Oct. 2021, doi: 10.1016/j.htct.2021.10.603.

[10] N. Sujatha, Lipeka D, Meghan Gowsick S, and Mohanraj S, “B POSITIVE: An AI-Based Blood Donation System,” *International Journal for Research Trends and Innovation (IJRTI)*, vol. 10, no. 11, pp. a250–a251, Nov. 2025, Accessed: Oct. 08, 2026. [Online]. Available: https://www.ijrti.org/papers/IJRTI2511034.pdf

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	HAMY Hettiarachchi	Registration No.	D/BSE/25/0026
Degree / Intake	SE / Intake 42	Group No.	53
Project Title	AI-Powered Emergency Blood Donor Recommendation and Prioritisation Decision Support System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
System Architecture & Three-Pillar AI Framework Design	Structured the hybrid architecture combining Deductive AI (rule-based reasoning), Predictive AI (machine-learning classification) and Decision/Explainable AI (MCDA and XAI). Designed the end-to-end flow from emergency request to verified candidate recommendations; prepared foundational methodology documentation.	Hybrid AI architecture; Python 3.10+; modular system design; Markdown	Final Report(architecture section); README.md; app.py About System page
Synthetic Donor Registry & Probabilistic Data Generation	Designed the synthetic-data generator for 1,400 donor records across 10 Sri Lankan municipal hubs. Defined demographic and blood-group distributions, response-rate assumptions, and deterministic random seeding (seed=42) to support reproducibility without using real personal health information.	Python; NumPy; Pandas; synthetic data modelling; random seed control	src/data_generator.py; data/donors.csv; data/blood_donor.db
Clinical Registry Analytics & Executive Dashboard Module	Developed the interactive dashboard with summary indicators for total donors, available donors, blood groups, average response rate, and saved emergency requisitions. Added visualizations for blood-group frequency, geographic distribution, age, and response patterns.	Streamlit; Matplotlib; Seaborn; custom CSS	app.py dashboard module; dashboard charts
Ethical Governance & Human-in-the-Loop Clinical Safeguards	Documented the system as an advisory decision-support tool requiring human-in-the-loop review. Described privacy protections, medical disclaimers, bias considerations, and the need for clinical staff oversight.	Healthcare AI ethics; HITL governance; clinical risk analysis	Final Report; app.py ethics/disclaimer section; README.md

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  Date: 17/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	RMSA Rathnayake	Registration No.	D/BCS/25/0026
Degree / Intake	CS / Intake 42	Group No.	53
Project Title	AI-Powered Emergency Blood Donor Recommendation and Prioritisation Decision Support System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
ABO/Rh blood compatibility engine	Designed the red-blood-cell compatibility matrix for all eight standard blood groups. Implemented forward donor matching and reverse recipient mapping using ABO/Rh antigen-antibody constraints.	Deductive AI; expert rules; serological knowledge modelling; Python	src/blood_compatibility.py; Final Report
Rule-based reasoning engine (Pillar 1)	Built the multi-stage filtering logic: blood compatibility, active availability, and a configurable minimum recovery interval (default 90 days). Included a fallback for cases where the recovery-interval filter leaves no candidates but available compatible donors remain.	Rule-based reasoning; constraint filtering; Pandas	src/rule_engine.py; STAGE2_PROGRESS_REVIEW.md
Emergency request workflow and audit funnel	Created the Emergency Request interface with recipient blood group, hospital, units required, urgency, safe-gap slider, and availability control. Added a funnel view showing how the candidate pool is reduced at each filtering stage and request-history logging.	Streamlit; reactive UI; SQLite logging; CSS cards	app.py Emergency Request module; src/database.py;
Sensitivity and hard-case testing	Tested different recovery intervals and validated the restrictive O-negative recipient case. Used assertions to check all 64 donor-recipient blood-group combinations and reviewed how changing safety settings affects candidate counts.	Boundary-value testing; sensitivity analysis; automated assertions	verify_pipeline.py; validation test suite; Final Report

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  Date: 17/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	WS Sanali Avihinsa	Registration No.	D/BIT/24/0079
Degree / Intake	IT / Intake 41	Group No.	53
Project Title	AI-Powered Emergency Blood Donor Recommendation and Prioritisation Decision Support System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Machine-learning response prediction (Pillar 2)	Built the supervised classification pipeline to estimate the probability that a donor will respond positively. Engineered features including distance, historical response rate, days since last donation, donation count, age, and availability; integrated preprocessing and real-time prediction.	Scikit-learn; feature engineering; StandardScaler; Haversine distance	src/train_model.py; src/preprocessing.py; src/prediction.py; model_training.ipynb
Algorithm benchmarking and cross-validation	Compared Random Forest, Logistic Regression, and Decision Tree models using a stratified 80/20 split and five-fold stratified cross-validation. Tuned the Random Forest and saved the selected model and evaluation metrics.	Classification; StratifiedKFold; hyperparameter tuning; Joblib	src/train_model.py; models/donor_response_model.pkl; models/model_metrics.pkl; report Section 9
Multi-criteria ranking and geographic trade-offs	Formulated a weighted score balancing response probability (40%), availability (30%), proximity (20%), and donation history (10%). Defined recommendation tiers and evaluated cases where a farther donor with a higher response probability may rank below a closer donor.	MCDA; weighted linear scoring; geographic optimization	src/ranking.py; Final Report
Model evaluation and limitations	Presented the comparison of Random Forest, Logistic Regression, and Decision Tree, and discussed the more realistic eligible-donor evaluation.	Classification metrics; ROC-AUC; F1-score	Model Performance page, benchmark table, confusion matrices, feature importance

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  Date: 17/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	AMSC Adhikari	Registration No.	6935
Degree / Intake	IT/ Intake 41	Group No.	53
Project Title	AI-Powered Emergency Blood Donor Recommendation and Prioritisation Decision Support System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Explainable AI engine and donor evidence cards	Developed the rationale generator that turns candidate data and scores into human-readable evidence. The six explanations cover ABO/Rh compatibility, availability, distance, historical response, predicted response probability, and donation history. Integrated expandable explanation cards so coordinators can inspect why a donor was recommended.	Explainable AI (XAI); rule-based natural-language generation; Streamlit	src/ranking.py generate_donor_explanation; app.py render_donor_card; Final Report
Transparent score breakdown and auditability	Implemented a score-deconstruction panel showing each weighted component and the final total. For the example donor D0149, the components are response probability 38.7, availability 30.0, proximity 17.5, and donation history 9.0, producing a total of 95.2/100. Documented that availability contributes the same 30 points to all candidates that pass the availability filter.	Multi-Criteria Decision Analysis; score normalization; transparent auditing	src/ranking.py calculate_component_scores; score breakdown in ranking module; report
Clinical action workflow and telephony simulation	Built simulated 'Send Emergency SMS' and 'Log Direct Call' actions on donor cards. The interface displays confirmation/log messages without sending real messages or incurring third-party communication costs.	Streamlit event handling; mock dispatch simulation; workflow design	app.py donor-card action buttons
Donor database and export	Developed the Donor Database page to inspect and filter the 1,400 synthetic records by blood group, city, availability, and age range. Added CSV export for filtered records and checked that the available O-negative count matches the rule-engine audit.	SQLite; Pandas filtering; Streamlit; CSV export	app.py show_database_page; src/database.py; CSV export utility

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  Date: 17/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.